Deep Q-Network Variants with Expert-Guided Stabilization for Autonomous Navigation in Dense Forest Environments

Author Name
School of xxx, xxx University
City, Country
xxx@xxx.edu

***Abstract*—Autonomous navigation in unstructured forest environments presents significant challenges due to dense obstacles, narrow passages, and the kinematic constraints of ground vehicles. This paper proposes a comprehensive Deep Q-Network (DQN) framework for forest navigation that integrates (i) six DQN variants spanning MLP and CNN architectures with standard, Double, and Polyak Double DQN update rules, (ii) a four-stage training pipeline incorporating expert demonstration prefilling, supervised pretraining, guided exploration with action masking, and temporal-difference learning with raw-reward replay, and (iii) a multi-level safety mechanism combining admissibility masks, top-k Q-value fallback, and short-horizon rollout heuristics. Extensive experiments on a procedurally generated forest map with Ackermann bicycle kinematics show that the proposed approach achieves a 97% success rate on long-range tasks ($42\,$m+) with a mean path length of $40.61\,$m—comparable to Hybrid A* ($40.86\,$m)—while reducing planning time by over $15\times$ ($0.25\,$s vs. $3.78\,$s). Ablation studies on reward shaping, speed constraints, and a DQN+MPC hybrid architecture provide insights into the design trade-offs for RL-based forest navigation.**

***Index Terms*—Deep reinforcement learning, autonomous navigation, forest environment, Deep Q-Network, path planning, action masking**

I. Introduction

Autonomous mobile robots (AMRs) operating in forest environments must navigate through dense, irregularly spaced obstacles while satisfying kinematic constraints of the vehicle platform. Classical planning algorithms such as Hybrid A* [1] and RRT* [2] provide completeness guarantees but incur high computational costs, especially in cluttered environments where frequent replanning is required.

Deep reinforcement learning (DRL) offers a promising alternative by learning reactive policies that map sensor observations directly to control actions. Among DRL methods, the Deep Q-Network (DQN) family [3] and its extensions—Double DQN [4] and soft target updates [5]—have demonstrated success in discrete-action domains. However, applying DQN to real-world navigation in forest-like environments poses several challenges:

1) **Long horizons**: Forest traversal episodes can span hundreds of time steps under fine-grained control (dt $= 0.05\,$s), causing reward signal dilution.
2) **Safety constraints**: The vehicle must avoid tree collisions under Ackermann steering kinematics with limited turning radius.
3) **Sparse rewards**: Binary success/failure feedback makes credit assignment difficult.
4) **Sample efficiency**: Pure exploration in dense forests leads to near-zero success rates early in training.

To address these challenges, we propose a comprehensive DQN-based navigation framework that combines: (a) a four-stage training pipeline with demonstration-guided bootstrapping, (b) a shaped reward function with multi-component penalties, (c) a layered action selection mechanism with safety guarantees, and (d) systematic comparison of six DQN variants across two neural network architectures.

The main contributions of this paper are:

- A complete DQN training framework for kinematically constrained forest navigation, incorporating DQfD-style pretraining, expert-guided exploration, and raw-reward replay with sampling-time normalization.
- Systematic evaluation of six DQN variants (MLP/CNN $\times$ DQN/DDQN/PDDQN) against Hybrid A* and RRT* baselines.
- Ablation studies quantifying the effects of goal-proximity reward shaping, terminal speed constraints, and DQN+MPC hybrid architectures.
- Demonstration that CNN-PDDQN achieves 100% long-range success with shorter paths and $15\times$ faster planning than Hybrid A*.

II. Related Work

A. Classical Path Planning

Hybrid A* [1] extends lattice-based planning with continuous steering angles, producing kinematically feasible paths. RRT* [2] provides asymptotic optimality but suffers from slow convergence in narrow passages. Both methods require explicit environment models and scale poorly with obstacle density.

B. Deep Reinforcement Learning for Navigation

DQN [3] introduced experience replay and target networks for stable Q-learning with deep neural networks. Double DQN [4] mitigates overestimation bias by decoupling action selection and evaluation. Polyak averaging (soft target

updates) [5] provides smoother target tracking. DQfD [6] integrates expert demonstrations via large-margin classification and temporal-difference losses.

Several works apply DRL to robot navigation. Tai *et al.* [7] trained DQN agents for mapless navigation using laser range data. Xie *et al.* [8] combined DRL with motion planning for outdoor navigation. However, most prior work targets indoor or urban environments; forest navigation with dense, irregularly distributed obstacles and bicycle kinematics remains underexplored.

C. Action Masking and Safety in RL

Invalid action masking [9] constrains the policy to feasible actions, improving sample efficiency and safety. Our work extends this with a multi-tier fallback system that combines admissibility checking, top-k Q-value search, and short-horizon rollout heuristics.

III. Problem Formulation

A. Vehicle Model

We model the vehicle as a front-steered bicycle with Ackermann kinematics:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \frac{v}{L}\tan\delta \tag{1}$$

where (x, y, θ) is the vehicle pose, v is the longitudinal velocity, δ is the front steering angle, and $L = 0.6\,\text{m}$ is the wheelbase. The dynamics are integrated with Euler's method at $\text{dt} = 0.05\,\text{s}$.

Vehicle constraints include maximum velocity $v_{\max} = 2.0\,\text{m/s}$ and maximum steering angle $\delta_{\max} = 27°$. Collision detection uses a two-circle footprint approximation with radius $r = 0.436\,\text{m}$ centered at $\pm 0.231\,\text{m}$ from the rear axle, as illustrated in Fig. 1.

B. Forest Environment

The forest map is procedurally generated using Poisson disk sampling with configurable tree count, spacing, and clearance parameters. We use the `forest_a` environment: a 360×360 cell grid at $0.1\,\text{m/cell}$ resolution (physical size: $36 \times 36\,\text{m}$) with 85 trees and a mean spacing of $3.0 \pm 0.15\,\text{m}$ (see Fig. 1). A seed-based generation ensures reproducibility across experiments.

C. MDP Formulation

The navigation task is formulated as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

State space $\mathcal{S}$: Each observation consists of an 11-dimensional scalar vector (vehicle pose, velocity, steering angle, goal pose, goal distance) concatenated with spatial map channels. MLP agents receive 2 downsampled map channels (occupancy grid + cost-to-go); CNN agents receive 3 channels (occupancy + cost-to-go + Euclidean Distance Transform clearance).

Action space $\mathcal{A}$: 35 discrete actions combining 7 steering rate levels $\dot{\delta} \in [-0.7, 0.7]$ rad/s and 5 acceleration levels $a \in [-1, 1]\,\text{m/s}^2$.

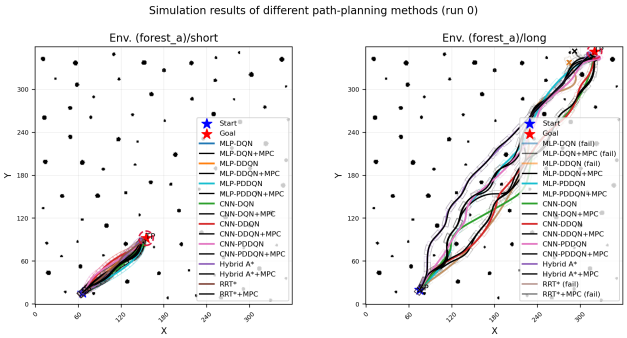


Fig. 1. Forest environment (`forest_a`) with planned paths from all algorithms. Left: short-range scenario (6–14 m); Right: long-range scenario (42 m+). Black dots represent trees; colored lines show paths from different algorithms.

TABLE I
DQN variant specifications.

Variant	Backbone	TD Target	Target Upd.
MLP-DQN	MLP 3×256	$\max_{a'} Q_{\theta^-}$	Hard/1000
MLP-DDQN	MLP 3×256	$Q_{\theta^-}(\arg\max Q_\theta)$	Hard/1000
MLP-PDDQN	MLP 3×256	$Q_{\theta^-}(\arg\max Q_\theta)$	Polyak τ=.005
CNN-DQN	CNN+FC	$\max_{a'} Q_{\theta^-}$	Hard/1000
CNN-DDQN	CNN+FC	$Q_{\theta^-}(\arg\max Q_\theta)$	Hard/1000
CNN-PDDQN	CNN+FC	$Q_{\theta^-}(\arg\max Q_\theta)$	Polyak τ=.005

Reward function: A multi-component shaped reward (detailed in Section IV-B).

Discount factor: $\gamma = 0.995$, chosen to propagate credit over the long horizons typical of forest traversal (300–600 steps per episode).

IV. Methodology

A. DQN Variant Family

We evaluate six DQN variants organized along two axes: network architecture (MLP vs. CNN) and update rule (DQN, Double DQN, Polyak Double DQN). Table I summarizes the key differences.

MLP backbone: A fully connected network with 3 hidden layers of 256 units each, ReLU activations, receiving the flattened observation vector (11 scalars + $2 \times N^2$ map pixels) as input.

CNN backbone: A three-layer convolutional encoder ($32 \to 64 \to 64$ channels, 3×3 kernels) processes the spatial map inputs (3 channels: occupancy, cost-to-go, EDT clearance). The CNN features are concatenated with the 11-dimensional scalar observation and fed into a 2-layer fully connected head (256 units each). The architecture is illustrated in Fig. 2.

PDDQN (Polyak Double DQN): Combines Double DQN's decoupled target computation with Polyak soft updates $\theta^- \leftarrow (1-\tau)\theta^- + \tau\theta$ at every training step, replacing the periodic hard replacement used by DQN and DDQN.

CNN Q-Network Architecture

Spatial branch:
Input: $3 \times N \times N$ maps
$\rightarrow$ Conv2d($3 \rightarrow 32$, 3×3, s=1, p=1) + ReLU
$\rightarrow$ Conv2d($32 \rightarrow 64$, 3×3, s=2, p=1) + ReLU
$\rightarrow$ Conv2d($64 \rightarrow 64$, 3×3, s=2, p=1) + ReLU
$\rightarrow$ Flatten

Scalar branch:
Input: 11-dim $(x, y, \theta, v, \delta, x_g, y_g, \theta_g, d, \ldots)$

Merge: Concat $\rightarrow$ FC(256)+ReLU $\rightarrow$ FC(256)+ReLU $\rightarrow$ FC(35) $\Rightarrow$ Q-values

Fig. 2. CNN Q-Network architecture. MLP variants omit the spatial branch and directly flatten all inputs.

TABLE II
REWARD FUNCTION COEFFICIENTS.

Symbol	Component	Value
k_p	Progress (cost-to-go reduction)	12.0
k_t	Time penalty (per step)	0.1
k_δ	Steering smoothness	1.5
k_a	Acceleration smoothness	0.2
k_κ	Curvature penalty	0.2
k_o	Obstacle proximity	1.5
k_v	Speed–obstacle coupling	2.0
	Collision terminal	-200
	Success terminal	$+400$

B. Reward Design

The per-step reward combines a progress signal with multiple penalty terms to encourage safe, smooth navigation:

$$\begin{aligned} r_t = & \underbrace{k_p \cdot (\text{cost}_{t-1} - \text{cost}_t)}_{\text{progress}} - \underbrace{k_t}_{\text{time}} - \underbrace{k_\delta (\Delta\delta)^2}_{\text{steering}} \\ & - \underbrace{k_a (\Delta a)^2 (v/v_{\max})^2}_{\text{accel.}} - \underbrace{k_\kappa \tan^2 \delta}_{\text{curvature}} \\ & - \underbrace{k_o \left[\frac{1}{d_o + \epsilon} - \frac{1}{d_s + \epsilon} \right]^+}_{\text{obstacle}} - \underbrace{k_v \cdot f_{\text{speed}}(d_o, v)}_{\text{speed}} \end{aligned} \quad (2)$$

where cost_t is the cost-to-go from the Hybrid A* heuristic map, d_o is the minimum obstacle distance, and d_s is the safety threshold. Terminal rewards are $+400$ for reaching the goal, -200 for collision, and a scaled penalty for timeout. Table II lists the coefficient values.

C. Four-Stage Training Pipeline

Our training pipeline addresses the cold-start problem through progressive bootstrapping. The four stages are summarized below and the complete pseudocode is given in Algorithm 1.

1) Stage 1: Expert Demonstration Prefilling: Hybrid A* generates kinematically feasible paths. We execute the expert policy in the environment, retaining only successful episodes. Approximately 5000 transitions are stored with the `demo=True` flag, ensuring they are protected from overwriting in the replay buffer.

Algorithm 1 Four-stage DQN training pipeline

Require: Environment $\mathcal{E}$, expert policy π_e, episodes T
Ensure: Trained Q-network Q_θ
Stage 0: Initialization
Init Q_θ, $Q_{\theta^-} \leftarrow Q_\theta$, replay buffer $\mathcal{D}$ (100K)
Init Welford normalizer $\mathcal{N}$
Stage 1: Expert prefill
while $|\mathcal{D}_{\text{demo}}| < 5000$ **do**
  Run π_e in $\mathcal{E}$ with random (s_0, g)
  if episode succeeds **then**
    Store transitions in $\mathcal{D}$ with $\texttt{demo} = \text{True}$
  end if
end while
Stage 2: Supervised pretraining
for $i = 1$ to 40,000 **do**
  Sample demo batch; update Q_θ with $\mathcal{L}_{\text{CE}} + \lambda_m \mathcal{L}_{\text{margin}}$
  if greedy policy reaches goal **then break**
  end if
end for
$Q_{\theta^-} \leftarrow Q_\theta$; save pretrain checkpoint
Stage 3: Main RL loop
for episode $= 1$ to T **do**
  Sample random (s_0, g) with $\text{cost}(s_0, g) \geq 6\,\text{m}$
  for $t = 1$ to max_steps **do**
    if rand() $< p_{\text{exp}}(\text{ep})$ **then**
      $a_t \leftarrow \pi_e(s_t)$ ▷ Expert exploration
    else
      $a_t \leftarrow \epsilon\text{-greedy}(Q_\theta, s_t, \text{mask}_t)$
    end if
    Execute a_t; store $(s_t, a_t, r_t^{\text{raw}}, s_{t+1}, d_t)$
    $\mathcal{N}$.update(r_t)
  end for
  TD updates with $\mathcal{L}_{\text{TD}} + \lambda_m \mathcal{L}_{\text{margin}} + \lambda_{\text{ce}} \mathcal{L}_{\text{CE}}$
end for
Stage 4: Model selection
Greedy-eval {final, best, pretrain} $\rightarrow$ save best

2) Stage 2: Supervised Pretraining (DQfD-style): For 40,000 gradient steps, we sample batches from the demo-only portion of the replay buffer and optimize:

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{CE}}(Q_\theta(s), a_{\text{expert}}) + \lambda_m \cdot \mathcal{L}_{\text{margin}} \quad (3)$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy between softmax Q-values and the expert action, and:

$$\mathcal{L}_{\text{margin}} = \max\left(0, \max_{a \neq a_e} Q(s, a) + m - Q(s, a_e)\right) \quad (4)$$

with margin $m = 0.8$. Pretraining is early-stopped if the greedy policy can reach the goal within 600 steps.

3) Stage 3: Main Training Loop: Each episode samples a random $(s_{\text{start}}, s_{\text{goal}})$ pair with minimum cost-to-go distance of 6 m. Action selection follows a two-tier scheme:

Tier 1 — Expert exploration: With probability $p_{\text{exp}}(t) = 0.7 \times (1 - t/(0.85 \cdot T))^+$, the Hybrid A* expert selects the action.

Tier 2 — ϵ-greedy with safety mask: Otherwise, with probability $\epsilon(t)$ a random admissible action is chosen; with probability $1 - \epsilon(t)$ the Q-network's greedy action is used, subject to admissibility checking and fallback (Section IV-D).

Raw-reward replay: Rewards are stored in their raw form in the replay buffer. A Welford online normalizer tracks the running mean and variance, and normalization is applied at

sampling time during TD updates. This prevents the distribution shift that occurs when normalized rewards are stored as the normalizer statistics evolve during training.

The combined loss per update step is:

$$\mathcal{L} = \mathcal{L}_{\text{TD}} + \lambda_m \mathcal{L}_{\text{margin}} + \lambda_{\text{ce}} \mathcal{L}_{\text{CE}} \tag{5}$$

where $\mathcal{L}_{\text{TD}}$ is the Smooth L1 (Huber) loss between predicted and target Q-values, and the demo losses only apply to transitions with `demo=True`. Gradients are clipped to $\|\nabla\| \leq 10$.

D. Multi-Level Safety Mechanism

At both training and inference time, a layered action selection pipeline ensures collision avoidance:

1) **Admissibility check**: For each candidate action, a short-horizon constant-action trajectory is simulated ($h = 15$ steps at inference, $h = 6$ during training). An action is admissible if: (a) no collision occurs, (b) minimum obstacle distance $\geq$ threshold, and (c) cost-to-go decreases by at least 10^{-4} m.
2) **Top-k fallback**: If $\arg\max Q$ is inadmissible, the top 10 Q-value actions are tested sequentially.
3) **Mask fallback**: If no top-k action passes, the full admissibility mask is computed and the highest-Q admissible action is selected.
4) **Rollout heuristic**: If all actions fail the progress check, a short-horizon rollout selects the action maximizing obstacle clearance.

During training, the mask uses `training_mode=True`, which relaxes the progress requirement (only checking collision safety), allowing the Q-network to learn forward progress from the reward signal.

V. Experimental Setup

A. Environment Configuration

All experiments use the `forest_a` environment (360×360 cells, 36×36 m, 85 trees, seed 101). Two evaluation scenarios are defined:

- **Short range** (6–14 m cost-to-go): Tests local obstacle avoidance and maneuverability.
- **Long range** (42 m+ cost-to-go): Tests global navigation capability across the full map.

B. Training Configuration

Table III summarizes the training hyperparameters.

C. Evaluation Protocol

Each algorithm is evaluated over 10 rollouts per scenario with fixed random seeds. The same $(s_{\text{start}}, s_{\text{goal}})$ pairs are shared across all algorithms and baselines to ensure fair comparison. Metrics include success rate, average path length (m), planning time (s), average curvature (m^{-1}), and composite score. Baselines include Hybrid A* (200K node budget) and RRT* (5K iterations), both with a 5 s timeout per query.

TABLE III
Training hyperparameters.

Parameter	Value
Episodes	1000
Max steps per episode	600
Optimizer	Adam
Learning rate	5×10^{-4}
Batch size	128
Replay buffer capacity	100,000
Discount factor γ	0.995
ϵ schedule	$0.9 \to 0.01$ (linear, 250 ep.)
Expert prob. schedule	$0.7 \to 0$ (linear, 85% of T)
Training freq.	1 update / 4 env. steps
Target update (hard)	every 1000 grad. steps
Target update (Polyak)	$\tau = 0.005$ per step
Gradient clip norm	10.0
Demo margin m	0.8
Demo loss weights	$\lambda_m = 1.0$, $\lambda_{\text{ce}} = 1.0$

VI. Results

A. Training Dynamics

Fig. 3 shows the training progress over 1000 episodes for all six DQN variants. The success rate converges to near 1.0 around episode 200 and remains stable. The average return settles around 400–500, consistent with successful episodes receiving the $+400$ terminal reward plus accumulated progress rewards.

Fig. 4 shows the episode reward curves during training. The transparent lines indicate raw per-episode returns, while the solid lines show moving averages. After an initial phase of high variance (episodes 0–50, corresponding to the expert exploration period), rewards stabilize around 400–500 for all variants.

Fig. 5 presents additional training diagnostics including total loss, TD loss, ϵ schedule, and Q-value statistics. The PDDQN variants (with Polyak updates) show notably smoother Q-value trajectories compared to the periodic hard-update variants.

B. Main Results

Table IV presents the complete evaluation results for all algorithms.

Success rate: The RL ensemble achieves 93% on short-range and 97% on long-range tasks. Four of six variants achieve 100% on long-range scenarios. MLP-PDDQN is the only variant reaching 100% success on *both* scenarios.

Path quality: CNN-PDDQN produces the shortest long-range paths (40.61 m), surpassing Hybrid A* (40.86 m) by 0.6%.

Computational efficiency: RL planning times range from 0.043 s to 0.275 s, compared to 1.29–3.78 s for Hybrid A*. MLP-PDDQN achieves the fastest planning (0.179 s on long-range), a **21**$\times$ speedup over Hybrid A*.

Path smoothness: RL agents produce smoother paths (curvature 0.10–0.18 m^{-1}) than RRT* (0.29–0.39 m^{-1}), comparable to Hybrid A*.

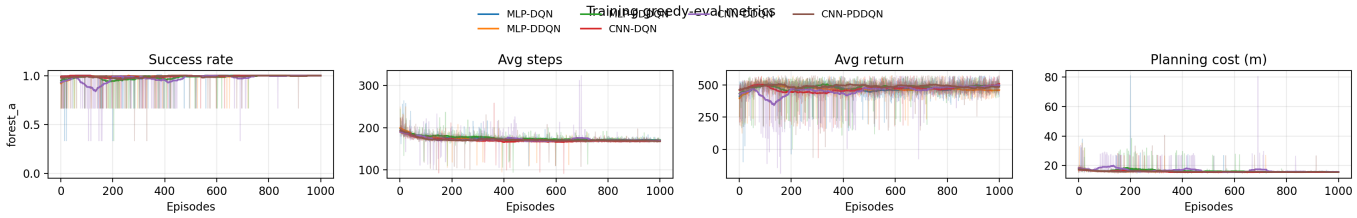


Fig. 3. Training evaluation metrics over 1000 episodes. From left to right: greedy success rate, average episode steps, average return, and planning cost. All variants converge to high success rates by episode 200.

TABLE IV
EVALUATION RESULTS ON FOREST_A (1000 EPISODES TRAINING). BEST RL RESULTS IN **BOLD**.

	Success Rate		**Path Length (m)**		**Plan Time (s)**		**Avg. Curv. (m^{-1})**	
Algorithm	Short	Long	Short	Long	Short	Long	Short	Long
MLP-DQN	0.9	0.9	8.84	44.14	0.079	0.200	0.112	0.153
MLP-DDQN	0.9	0.9	8.84	44.14	0.064	0.238	0.112	0.153
MLP-PDDQN	**1.0**	**1.0**	8.40	**41.17**	**0.043**	**0.179**	0.175	0.105
CNN-DQN	0.9	**1.0**	8.32	41.39	0.076	0.275	0.117	0.112
CNN-DDQN	0.9	**1.0**	8.42	41.12	0.080	0.225	0.146	0.120
CNN-PDDQN	0.9	**1.0**	**8.31**	**40.61**	0.070	0.254	0.167	**0.104**
RL Mean	*0.93*	*0.97*	—	—	—	—	—	—
Hybrid A*	1.0	1.0	8.39	40.86	1.291	3.782	0.116	0.100
RRT*	1.0	0.9	9.14	44.04	0.255	1.804	0.386	0.286

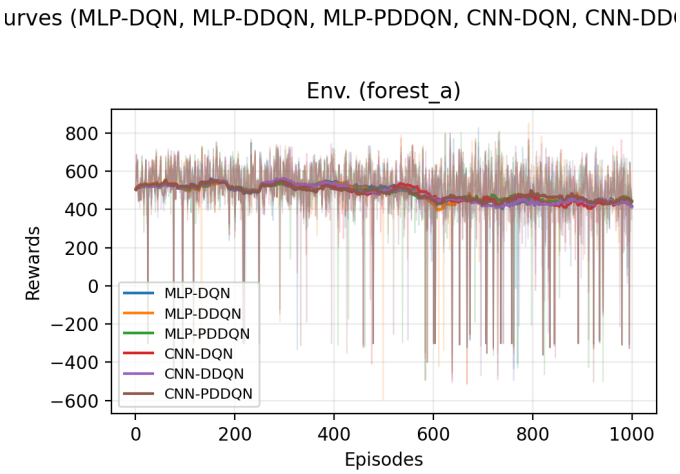


Fig. 4. Training reward curves (1000 episodes). Transparent lines: raw episode returns; solid lines: moving average. All six variants converge to similar reward levels (∼400–500).

C. Path Visualization

Fig. 6 shows the planned paths from all algorithms on both evaluation scenarios. The RL agents produce smooth, near-optimal trajectories that closely follow the Hybrid A* paths while avoiding all obstacles.

D. Control Profiles

Fig. 7 shows the speed and steering angle profiles during inference. RL agents make frequent small steering adjustments to navigate through tree gaps, while MPC-based methods exhibit larger oscillations.

TABLE V
ABLATION: GOAL-PROXIMITY REWARD SHAPING. RL MEAN SUCCESS RATE ACROSS 6 VARIANTS (300 EPISODES).

	A: Base	**B: +Shape**	**C: +Spd1.0**	**D: +Spd0.5**
Short	0.92	0.88	0.72	0.67
Long	0.95	0.87	0.43	0.33

E. Architecture Comparison

MLP vs. CNN: CNN variants achieve uniformly 100% success on long-range tasks, while MLP-DQN and MLP-DDQN reach only 90%. The EDT clearance channel (exclusive to CNN) provides valuable spatial awareness for long-range navigation.

DQN vs. DDQN vs. PDDQN: PDDQN (Polyak soft updates) outperforms both DQN and DDQN variants. MLP-PDDQN is the only MLP variant achieving 100% on both scenarios.

F. Ablation Studies

1) Goal-Proximity Reward Shaping: Table V shows the effect of adding a goal-proximity bonus to the base reward. Four variants are compared: (A) baseline without shaping, (B) goal-proximity shaping ($k_{\text{goal}} = 1.5$), (C) shaping + loose speed constraint ($v_{\text{goal}} \leq 1.0$ m/s), and (D) shaping + strict speed constraint ($v_{\text{goal}} \leq 0.5$ m/s).

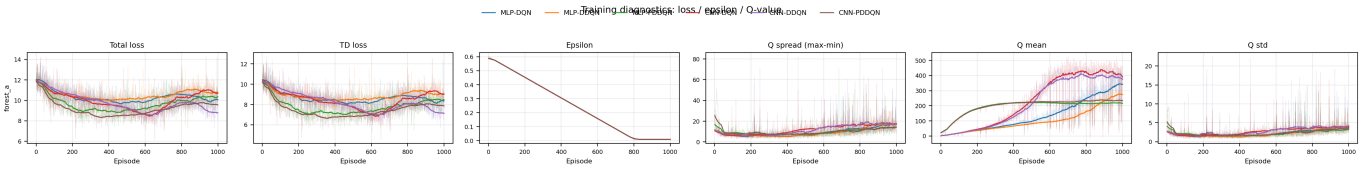


Fig. 5. Training diagnostics. From left to right: total loss, TD loss, ϵ schedule, Q-value spread (max$-$min), Q-value mean, and Q-value standard deviation. PDDQN variants exhibit smoother Q-value dynamics due to Polyak soft updates.

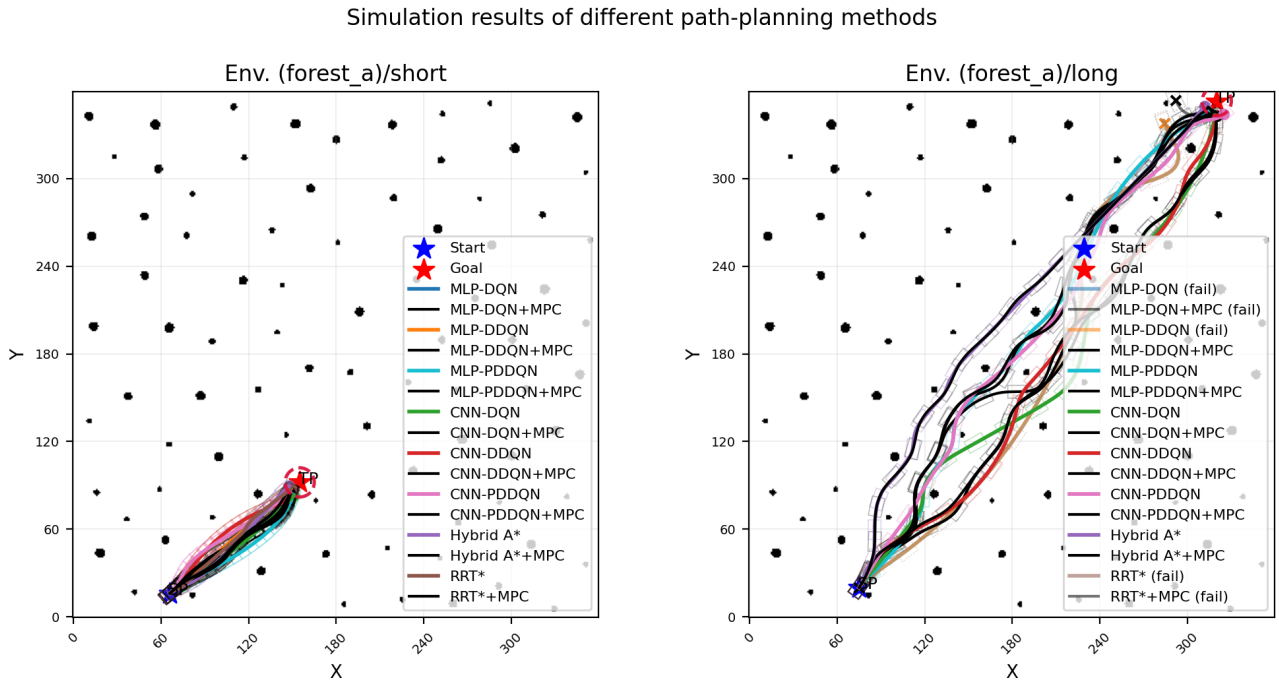


Fig. 6. Planned paths from all algorithms. Left: short-range; Right: long-range. RL agents (colored lines) produce smooth paths comparable to Hybrid A* (black dashed), while RRT* paths (gray) show higher curvature.

TABLE VI
PER-VARIANT SUCCESS RATES UNDER GOAL-PROXIMITY SHAPING (SHORT / LONG).

	A: Baseline		B: +Shaping	
Variant	Short	Long	Short	Long
MLP-DQN	0.8	0.8	**1.0**	**1.0**
MLP-DDQN	0.9	0.9	0.9	0.9
MLP-PDDQN	**1.0**	**1.0**	0.9	0.9
CNN-DQN	0.9	**1.0**	0.9	0.7
CNN-DDQN	0.9	**1.0**	**1.0**	**1.0**
CNN-PDDQN	**1.0**	**1.0**	0.6	0.7

Finding 1: Goal-proximity shaping provides marginal gains for some variants (MLP-DQN: $0.8 \rightarrow 1.0$) but degrades others (CNN-PDDQN: $1.0 \rightarrow 0.6$).

Finding 2: Speed constraints at the goal dramatically reduce success rates. At 1.0 m/s tolerance, long-range success drops from 0.87 to 0.43; at 0.5 m/s, it collapses to 0.33.

Fig. VI shows the per-variant breakdown.

Fig. 8 compares the training curves across the four ablation variants.

TABLE VII
ABLATION: PURE DQN VS. DQN+MPC HYBRID. RL MEAN SUCCESS RATE (300 EPISODES).

	Pure DQN	DQN+MPC
Short	0.92	0.77 (-16%)
Long	0.95	0.87 (-8%)

2) DQN + MPC Hybrid Architecture: Table VII compares pure DQN control against a two-stage architecture where DQN generates a global path and MPC tracks it.

The DQN+MPC hybrid consistently underperforms pure DQN control. Table VIII shows the per-variant analysis: MLP variants show slight improvement on long-range tasks, while CNN variants degrade significantly.

3) Effect of Training Duration: Extending training from 300 to 1000 episodes (Table IX) improves long-range success from 0.95 to 0.97 without degradation on short-range tasks.

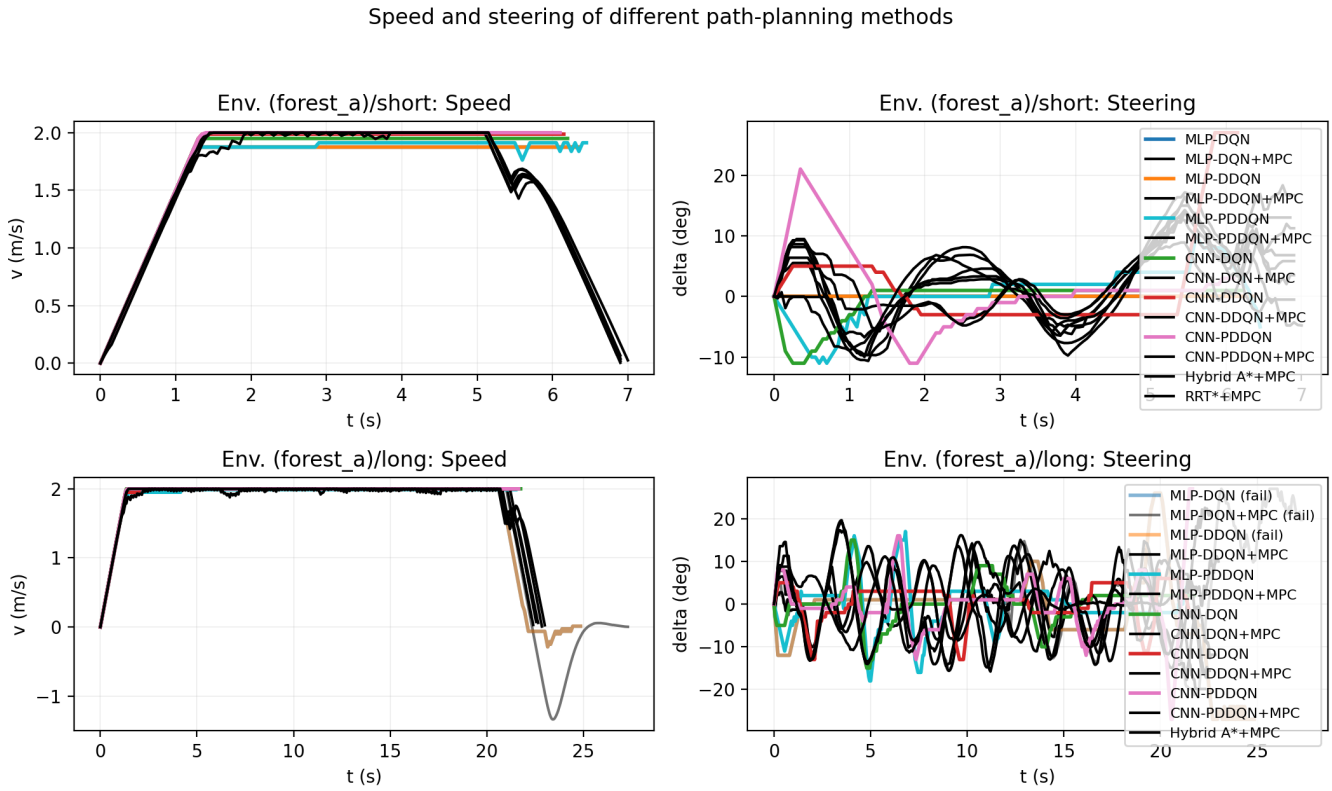
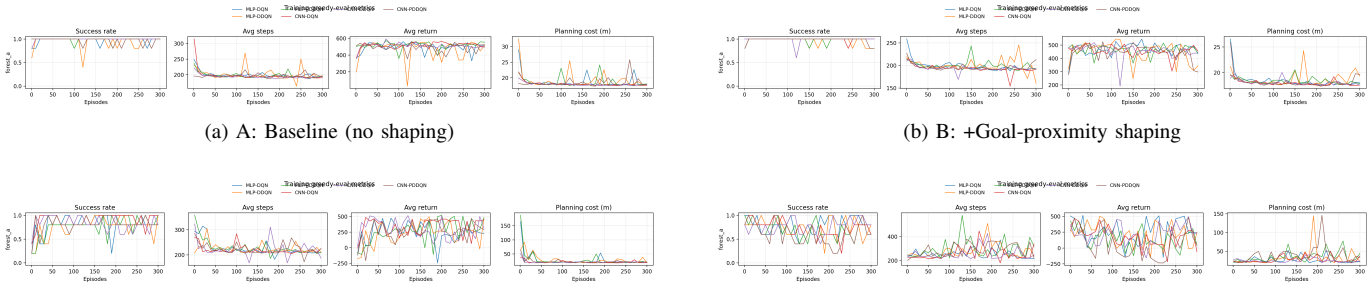


Fig. 7. Speed and steering angle profiles during inference. Top row: short-range; Bottom row: long-range. RL agents show smooth velocity profiles with rapid acceleration and gradual deceleration.



(a) A: Baseline (no shaping)

(b) B: +Goal-proximity shaping

(c) C: +Speed constraint (1.0 m/s)

(d) D: +Speed constraint (0.5 m/s)

Fig. 8. Training evaluation metrics for the four reward shaping ablation variants (300 episodes each). The baseline (A) and shaping-only (B) converge stably, while speed constraints (C, D) cause severe performance degradation.

VII. Discussion

RL vs. classical planners. Our results demonstrate that DQN-family agents can match or exceed Hybrid A* in path quality while providing over an order of magnitude faster computation. This speed advantage is critical for real-time navigation where replanning frequency matters. However, RL agents do not yet achieve the 100% reliability of Hybrid A* across all variants, suggesting that safety-critical applications may benefit from a hybrid approach using RL for fast initial planning with classical planner verification.

The role of expert guidance. The four-stage pipeline is essential for forest navigation. Without demo prefilling and pretraining, agents fail to learn meaningful policies within feasible training budgets. The expert probability decay ($0.7 \rightarrow 0$ over 85% of training) provides a smooth transition from imitation to autonomous policy.

Raw-reward replay. Storing raw rewards and normalizing at sampling time eliminates the distribution shift problem inherent in normalizing before storage. As the normalizer statistics evolve during training, previously stored normalized values become stale; raw storage avoids this inconsistency.

Safety mechanism trade-offs. The multi-level fallback system trades exploration for safety. In training mode, relaxing the progress requirement allows the Q-network to discover novel

TABLE VIII
DQN+MPC PER-VARIANT SUCCESS RATES (SHORT / LONG).

	Pure DQN		DQN+MPC	
Variant	Short	Long	Short	Long
MLP-DQN	0.8	0.8	0.7	0.9
MLP-DDQN	0.9	0.9	0.6	1.0
MLP-PDDQN	1.0	1.0	0.8	1.0
CNN-DQN	0.9	1.0	0.8	0.8
CNN-DDQN	0.9	1.0	0.8	0.7
CNN-PDDQN	1.0	1.0	0.9	0.8

TABLE IX
EFFECT OF TRAINING DURATION ON RL MEAN SUCCESS RATE.

	300 episodes	1000 episodes
Short	0.93	0.93
Long	0.95	0.97 (+2%)

strategies, while full safety checking at inference prevents collision.

Why DQN+MPC fails. The failure of the DQN+MPC hybrid reveals a fundamental asymmetry: classical planners guarantee path completeness, making MPC tracking reliable. DQN paths, generated by sequential reactive decisions, may contain loops, dead ends, or fail to reach the goal entirely. MPC tracking such paths amplifies rather than corrects these structural deficiencies.

Limitations and future work. (1) Evaluation is limited to a single forest configuration; generalization to denser forests needs further study. (2) The 35-action discrete space may limit fine-grained control; continuous action variants (DDPG, SAC) could improve path smoothness. (3) Goal speed constraints proved infeasible with 300 episodes; specialized deceleration curricula remain an open challenge. (4) Sim-to-real transfer has not been addressed.

VIII. CONCLUSION

We presented a comprehensive DQN-based framework for autonomous navigation in dense forest environments. Through a four-stage training pipeline combining expert demonstration, supervised pretraining, guided exploration, and temporal-difference learning, our approach achieves up to 100% success on long-range navigation tasks with Ackermann bicycle kinematics. CNN-PDDQN emerges as the best variant, producing paths of 40.61 m—shorter than Hybrid A* (40.86 m)—while computing over $15\times$ faster (0.25 s vs. 3.78 s). Systematic ablation studies reveal three key insights: (i) goal-proximity reward shaping offers limited and inconsistent benefits, (ii) terminal speed constraints require substantially more training than goal-reaching alone, and (iii) DQN+MPC hybrids underperform direct RL control due to path completeness issues. Future work will extend the framework to multi-map generalization, continuous action spaces, and sim-to-real transfer.

REFERENCES

[1] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," in *The International Journal of Robotics Research*, vol. 29, no. 5. SAGE Publications, 2010, pp. 485–501.

[2] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[5] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[6] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband *et al.*, "Deep Q-learning from demonstrations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[7] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 31–36.

[8] L. Xie, S. Wang, A. Markham, and N. Trigoni, "Learning with training wheels: Speeding up training with a simple controller for deep reinforcement learning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 6276–6283.

[9] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *The International FLAIRS Conference Proceedings*, vol. 35, 2022.